

Submission Worksheet

IT202-450-M2026 / it202-milestone-3-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 3 - 2026
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100%
Potential Grade	10.00/10.00 (100.00%)
Received Grade	0.00/10.00 (0.00%)
Updated	8/9/2026, 2:50:25 AM
Grading Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-3-2026/grading/mrc82
View Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-3-2026/mrc82

Instructions

Verify that the lesson-built relationship layer is complete and functional, then collect focused evidence for the association design, logged-in user view, cross-user and unassociated views, assignment workflow, and public profile requirements. Most coding should already be complete from the lessons; this worksheet focuses on confirming the full relationship flow, fixing remaining issues, and explaining your decisions.

Assignment Setup

- Required branch: `Milestone3`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
 - Read the whole worksheet.
 - Review the modern project proposal and Milestone 3 assignment requirements.
 - Check out `qa`, pull the latest version, and create the milestone branch.
 - Complete and test the relationship-table and association lessons.
 - Merge the milestone branch into `qa`, test deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the milestone branch with `git checkout -b Milestone3`.
4. Commit focused relationship and interface work in small pieces.
5. Push `Milestone3`.
6. Open a pull request from `Milestone3` into `qa` and keep it open while you work.
7. After all Milestone 3 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.

6. Open a pull request from `Milestone3` into `qa` and keep it open while you work.
7. After all Milestone 3 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.
9. Export the worksheet PDF from the Learn Courses Platform.
10. Save the PDF in the repository under `docs/milestone03/`.
11. Add, commit, and push the PDF to the `Milestone3` branch.
 - `git add .`
 - `git commit -m "Add milestone 3 worksheet PDF"`
 - `git push origin Milestone3`
12. Upload the same PDF to Canvas.
13. Create and merge a pull request from `qa` into `prod` when production evidence is required.
14. Switch back to `qa`.
15. Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 3 feature:

- Add a comment with your UCID.
- Add the date to the comment.
- Briefly summarize the code block or feature area in the comment.
- Load shared project setup through `lib/app.php`.
- Follow the database-helper patterns from the lessons.
- Follow the validation patterns from the lessons.
- Follow the flash-message patterns from the lessons.
- Follow the session patterns from the lessons.
- Follow the role-check patterns from the lessons.
- Use a relationship table when many users can connect to many project records.
- Include `id` in the relationship table.
- Include the user foreign key in the relationship table.
- Include the project-record foreign key in the relationship table.
- Include `created` in the relationship table.
- Include `modified` in the relationship table.
- Document whether repeated user/project-record pairs are valid for the project's relationship model.
- Prevent unintended repeated associations with server-side validation.
- A database uniqueness constraint is recommended when repeated pairs are not valid.
- When repeated pairs are valid, store values that distinguish the purpose of each relationship row.
- Remove or deactivate relationships without accidentally deleting the related user or project record.
- Use prepared statements or shared database helpers for every relationship query.
- Enforce the project's selected role or ownership rule on the server for association creation.
- Enforce the project's selected role or ownership rule on the server for remove-one actions.
- Enforce the project's selected role or ownership rule on the server for remove-all actions.
- Enforce role rules on the server for protected lists.
- Enforce role rules on the server for assignment actions.
- Document whether the all-associated view is public or role-protected.
- Enforce the all-associated visibility decision on the server.
- Document whether the unassociated view is public or role-protected.
- Enforce the unassociated visibility decision on the server.
- Validate list controls on the server.
- Keep limits from 1 through 100.
- Default invalid limits to 10.
- Keep valid filters sticky.
- Keep valid sorting choices sticky.
- Provide at least one entity-specific filter on each required list.

- Keep valid filters sticky.
- Keep valid sorting choices sticky.
- Provide at least one entity-specific filter on each required list.
- Provide at least one selectable sort option in addition to the default sort.
- Show a clear no-results message instead of an empty or broken result layout.
- Show the total number of records matching the active filters where the project proposal requires it.
- Show the number of records displayed on the current page where the project proposal requires it.
- Summarize each entity with its title or identifier and at least one additional project-specific field used in Milestone 2.
- Summarize each relationship with its required `id`, `created`, and `modified` columns.
- A relationship summary may also include status or another project-specific column.
- Show usernames on public pages without exposing private account values:
 - Email
 - Roles
 - Session data
 - Other private account values
- Use POST for state-changing association actions.
- Use user-friendly flash messages.
- Log technical errors with `error_log()`.
- Apply project styling to every page.
- Apply consistent navigation to every page.
- Test through supported deployed `qa` URLs before collecting browser evidence.
- Use your own relationship concept.
- Use your own project data.
- Use your own database behavior.
- Use your own code.
- Use your own branch.
- Use your own pull request.
- Use your own deployed URLs.
- Use your own explanations.

Evidence Format

Most sections use the same evidence pattern:

- Images:
 - Show relevant code and matching browser or database output.
 - Make sure UCID/date comments are visible in code evidence.
 - Make sure the deployed `qa` URL is visible in browser evidence.
 - One combined screenshot is acceptable when both parts remain readable.
 - Reuse evidence when one image proves requirements in more than one section.
 - Use GitHub URLs for complete files and screenshots for the focused logic requested by the task.
 - Do not repeat shared helper code after the task that first verifies it.
- Grading:
 - Each Details bullet is required for full credit in that evidence field.
 - Partial credit is based on the listed requirements that are clearly demonstrated.
 - When the proposal permits a project decision, state the decision and show evidence that the application follows it.
- URLs:
 - Provide direct GitHub file links from the milestone branch.
 - A GitHub file URL must end with the file's expected extension; zero credit for that URL entry if it does not.
 - Provide anticipated production URLs for deployed PHP pages.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - Production page URLs must include `-prod` and end in `.php`; zero credit for that URL entry if they do not.

- Provide anticipated production URLs for deployed PHP pages.
- Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
- Production page URLs must include `-prod` and end in `.php`; zero credit for that URL entry if they do not.
- Response:
 - Explain the logic and decisions in your own words.
 - Describe what the relevant PHP, SQL, validation, role check, or query does.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the `Milestone3` branch has been pushed.
- Confirm the pull request into `qa` has been pushed.
- Confirm the deployed `qa` site was reviewed before collecting evidence.
- Test association creation.
- Test repeated-association behavior according to the documented project rule.
- Test remove-one behavior.
- Test remove-all behavior.
- Test the logged-in user view.
- Test the all-associated view.
- Test the unassociated view.
- Test the assignment view.
- Test the public-profile view.
- Test server-side ownership or role protection.
- Test one friendly failure state.
- Test relevant filters.
- Test sorting.
- Test valid limits.
- Test invalid limits.
- Test no-results states.
- Test matching counts.
- Test displayed counts.
- Confirm every submitted page URL uses the production deployment.
- Confirm every submitted page URL uses the same path as the tested `qa` URL.
- Confirm every GitHub file URL ends with its expected file extension.
- Commit the worksheet PDF under `docs/milestone03/`.
- Upload the same PDF to Canvas.

Submission Progress

100%

Section #1: (2 pts.) Association Design And Behavior

100%

- Goal: show what the user-to-project relationship means and how the database protects it.
- Expected behavior:
 - The relationship table includes a user foreign key.
 - The relationship table includes a project-record foreign key.
 - Foreign-key constraints prevent orphan relationships.
 - The project documents whether repeated user/project-record pairs are valid.
 - Repeated pairs include distinguishing relationship values when they are valid.
 - At least one workflow permitted by the project's selected role or ownership rule creates associations.
 - The server handles a repeated association without an unhandled error.
 - Removing an association leaves the user and project record intact.
 - The response states what users see when an associated record is updated manually or through the API.

- The server handles a repeated association without an unhandled error.
- Removing an association leaves the user and project record intact.
- The response states what users see when an associated record is updated manually or through the API.

📦 Task #1 (1 pt.) – Relationship Table Evidence

100%

📦 Combo #1

🖼️ Part 1: Images 100%

📋 Details

- Show the relationship table structure with `id`, both foreign keys, `created`, `modified`, and foreign-key constraints visible.
- Show several relationship rows connecting users and project records.
 - When repeated pairs are valid, show the values that distinguish their purpose.
- Show the focused SQL for the relationship rules:
 - Both foreign-key definitions
 - Any optional uniqueness rule used by the project



UserCharacters table structure with unique user-character pairs and both foreign-key constraints.



UserCharacters rows connecting users to saved character records.

🔗 Part 2: URLs 100%

📋 Details

- Paste the direct GitHub file link for the relationship-table SQL.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/sql/008_create_table_usercharacters.sql 🍏

✍️ Part 3: Response 100%

📋 Details

- In 3–5 sentences, explain:
 - What one relationship row means
 - How it connects both parent records
 - How the project prevents or distinguishes repeated pairs

Response

So each UserCharacters row represents one saved-character relationship between a user and a character. The `user_id` connects the row to Users, while `character_id` connects it to Characters. Repeated user-character pairs are not allowed because the table has a unique constraint on `user_id` and `character_id`. Instead of creating duplicates, the `is_active` value can be changed to activate or deactivate the existing relationship.

Task #2 (1 pt.) – Association Behavior Evidence

100%

Combo #1

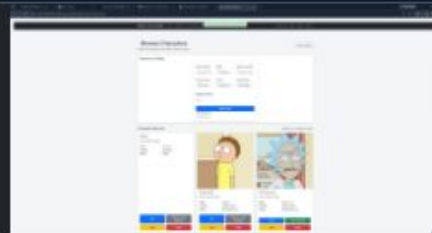
Part 1: Images 100%

Details

- Show one successful association result on the deployed qa site:
 - The submitted action
 - The resulting relationship row
- Show the project's documented result when the same association is submitted again:
 - A friendly rejection
 - Reactivation or deactivation of the existing relationship
 - A distinct row with a distinguishing value when repeated pairs are valid
- Show focused handler code containing:
 - The association write
 - The server-side access check



Successful character association created on the deployed QA site.



Existing character association deactivated on the deployed QA site.



Handler blocks unauthenticated users before changing saved-character relationships.



Handler inserts or updates the existing UserCharacters relationship instead of creating duplicates.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the association handler or page.
- Paste the anticipated production URL for the page where the tested association begins.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/internal/toggle_saved_character.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/characters.php> 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:

① Details

- In 3–5 sentences, explain:
 - How the workflow creates an association
 - How it handles an existing pair
 - How removal preserves both parent records
 - What associated views show after a record is edited manually or refreshed from the API

Response

Saving a character creates a UserCharacters relationship between the logged-in user and that character. If the same pair already exists, the workflow updates its `is_active` value instead of creating a duplicate row. Removing a saved character only deactivates the relationship, so the user and character records remain intact. Because the relationship points to the existing character record, associated views show the current character details after manual edits or API updates.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

478 chars

Section #2: (2 pts.) Logged In User Associations

100%

- Goal: show that a logged-in user can browse and manage only the associations permitted by the project's selected ownership rule.
- Expected behavior:
 - The page loads project records associated with the current user's session ID.
 - Each item shows the entity title or identifier and at least one additional project-specific entity field used in Milestone 2.
 - Each item shows the relationship `id`, `created`, and `modified` values.
 - Each item links to the project-record detail page.
 - A user permitted by the project's selected ownership rule can remove one association.
 - A user permitted by the project's selected ownership rule can remove all of their associations.
 - The page includes at least one entity-specific filter.
 - The page includes at least one selectable sort option in addition to the default sort.
 - The page accepts limits from 1 through 100 and defaults invalid values to 10.
 - The page shows an accurate matching count.
 - The page shows an accurate displayed count.
 - The page shows a clear no-results message when no records match.

🔗 Task #1 (1 pt.) – User Association List Evidence

100%

🔗 Combo #1

🖼️ Part 1: Images 100%

① Details

- Show the deployed `qa` list with these items visible:
 - Entity summaries
 - Relationship values
 - Detail links
 - Filter controls
 - Sort controls
 - Limit control
 - Matching count
 - Displayed count
- Show both list states:
 - One non-default filter or sort result
 - One clear no-results state
- Show focused code for:
 - The current-user query

- One clear no-results state
- Show focused code for:
 - The current-user query
 - Shared list-state handling
 - Limit validation
 - Matching-count logic



Saved-character list with filters, sorting, counts, and relationship details on QA.



Filtered saved-character view showing one matching Morty result on QA.



No-results state showing zero matching saved characters on QA.



Saved-character page uses the logged-in user ID and shared filtering and sorting controls.



Saved-character query restricts results and counts to the current logged-in user.



Result-limit validation allows 1–100 records and defaults invalid values to 10.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the logged-in user's association list.
- Paste the anticipated production URL for the logged-in user's association page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/my_characters.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/my_characters.php 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How the query limits records to the logged-in user
 - How the list controls change the query
 - How the matching count is calculated
 - How the displayed count is calculated

- How the displayed count is calculated

Response

The query limits saved characters to the current logged-in user by using the session user ID in the UserCharacters query. The filter, sort, and limit controls change the SQL conditions and ordering used for the saved-character list. The matching count is calculated from all active saved-character rows that match the selected filters for that user. The displayed count is the number of records shown on the current page after the limit and pagination are applied.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

464 chars

Task #2 (1 pt.) – User Association Removal Evidence

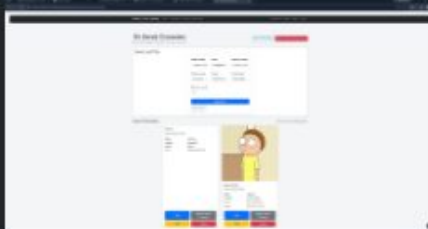
100%

Combo #1

Part 1: Images 100%

Details

- Show the deployed **qa** relationship state before and after one successful remove-one action.
- Show that the user account and project record still exist after the remove-one action.
- Show the deployed **qa** relationship state before and after one successful remove-all action.
- Show focused code containing the ownership check for each action.



Saved-character relationships before removing one association on QA.



One saved-character association removed while the remaining relationship stays intact.



Morty Smith still exists after the saved-character relationship was removed.



Remaining saved-character relationship before using Remove All on QA.



All saved-character associations removed while the user account remains intact.



Remove-one action updates only the logged-in user's saved-character relationship.



Remove-all action deactivates only the logged-in user's saved-character relationships.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the remove-one handler.
- Paste the direct GitHub file link for the remove-all handler.
- Paste the anticipated production URL for the page where the removal actions begin.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/internal/toggle_saved_character.php 🍏

URL #2

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/internal/clear_saved_characters.php 🍏

URL #3

https://mrc82-it202-450-m2026-prod.onrender.com/project/my_characters.php 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How both handlers verify the current user's ownership
 - How both handlers limit their changes to relationship rows

Response

The remove-one handler uses the logged-in session user ID and applies the change only to that user's UserCharacters relationship. The remove-all handler also uses `get_user_id()` and updates only rows where `user_id` matches the current user. Both actions change the `is_active` value in UserCharacters instead of deleting the related user or character records. This keeps the parent records intact while safely removing the saved-character relationships.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

449 chars

Section #3: (1 pt.) All Associated Records

100%

- Goal: show relationship rows across multiple users with the project's selected visibility rule.
- Expected behavior:
 - Each row shows the associated username.
 - Each row shows the entity title or identifier and at least one additional project-specific entity field used in Milestone 2.
 - Each row shows the relationship `id`, `created`, and `modified` values.
 - Each username links by user ID to a safe public profile.
 - Each project record links to its detail page.
 - A rule permitted by the project's selected removal rule can remove an individual relationship.

- Each username links by user ID to a safe public profile.
- Each project record links to its detail page.
- A role permitted by the project's selected removal rule can remove an individual relationship.
- A partial username filter is available.
- At least one entity-specific filter is available.
- At least one selectable sort option is available in addition to the default sort.
- The view accepts limits from 1 through 100 and defaults invalid values to 10.
- The view shows an accurate matching count.
- The view shows an accurate displayed count.
- The view shows a clear no-results message when no records match.
- The project documents whether this view is public or role-protected.
- The server enforces the documented visibility decision.

🔧 Task #1 (0.5 pts.) – All-Associated List Evidence

100%

🔧 Combo #1

🖼️ Part 1: Images 100%

① Details

- Show the deployed **qa** view with these items visible:
 - Usernames
 - Entity summaries
 - Relationship values
 - Public-profile links
 - Entity-detail links
 - Filter controls
 - Sort controls
 - Limit control
 - Matching count
 - Displayed count
- Show both list states:
 - One partial-username or entity-filter result
 - One clear no-results state
- Show focused code containing:
 - The relationship join
 - The documented visibility check



Character Associations page showing a clear zero-results state on QA.



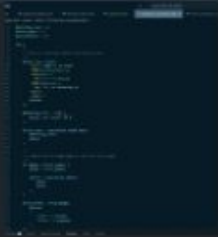
Active character relationships across multiple users with filters, counts, links, and relationship details.



Username filter showing one matching relationship and updated result count.



Character Associations page showing zero matches for an unmatched username filter.



Relationship query joins UserCharacters with Users and Characters for the associated-record list.



All-associated view is restricted to users with the Admin role.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the all-associated page.
- Paste the anticipated production URL for the all-associated view.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/admin/character_associations.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/character_associations.php 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How the join produces one row per relationship
 - How filters and counts stay aligned
 - What the selected visibility rule is
 - How the server enforces that rule

Response

The join connects UserCharacters to Users and Characters, so each relationship row includes the related username and character details. Filters narrow the matching rows, while the matching count reflects all filtered results and the displayed count reflects the rows currently shown. The selected visibility rule is Admin-only access for the all-associated view. The server enforces this with `require_role("Admin")` before the page content is shown.

Supports **bold**, *italic*, [links](#)(url), ``code``, etc.

448 chars

Task #2 (0.5 pts.) – All-Associated Removal Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the deployed **qa** relationship state before and after one successful authorized removal.
- Show one rejected unauthorized removal attempt.
- Show focused handler code containing:
 - The server-side role check
 - The relationship removal logic

- Show one or more unauthorized removal attempts
 - The server-side role check
 - The relationship-removal query



Authorized Admin removal successfully removed one relationship while the other remained.



Unauthorized user is redirected to login before accessing Admin relationship removal.



Removal handler enforces Admin access before processing relationship removal.



Removal handler deactivates only the selected active UserCharacters relationship row.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the relationship-removal handler.
- Paste the anticipated production URL for the page where the removal begins.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/internal/remove_character_association.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/character_associations.php 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How the server limits removal to the role selected by the project
 - How the request changes only the relationship record

Response

The server limits removal to Admin users by requiring the Admin role before the handler can run. The handler finds the selected active UserCharacters relationship by its relationship ID. It then changes only that relationship row by setting `is_active` to 0. The related user and character records are not deleted or changed.

Supports **bold**, *italic*, [links](url), ``code``, etc.

323 chars

Section #4: (1 pt.) Unassociated Records

100%

Section #4: (1 pt.) Unassociated Records

100%

- Goal: show project records that currently have no user relationship.
- Expected behavior:
 - The view excludes every project record that has at least one user relationship.
 - Each item shows the entity title or identifier and at least one additional project-specific entity field used in Milestone 2.
 - Each item links to its detail page.
 - At least one entity-specific filter is available.
 - At least one selectable sort option is available in addition to the default sort.
 - The view accepts limits from 1 through 100 and defaults invalid values to 10.
 - The view shows an accurate matching count.
 - The view shows an accurate displayed count.
 - The view shows a clear no-results message when no records match.
 - The project documents whether this view is public or role-protected.
 - The server enforces the documented visibility decision.

Task #1 (1 pt.) – Unassociated Records Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the deployed **qa** view with these items visible:
 - Entity summaries
 - Detail links
 - Filter controls
 - Sort controls
 - Limit control
 - Matching count
 - Displayed count
- Show both list states:
 - One non-default filter or sort result
 - One clear no-results state
- Show focused code containing:
 - The unassociated query or subquery
 - The documented visibility check



Unassociated character list showing records, detail links, filters, sorting, limits, and counts on QA.



Name filter showing one matching unassociated character and updated result count.



Unassociated Characters page showing zero matches for an unmatched name filter.



Unassociated query excludes characters that already have an active UserCharacters relationship.



Unassociated-records view is restricted to users with the Admin role.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the unassociated page.
- Paste the anticipated production URL for the unassociated view.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/admin/unassociated_characters.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/unassociated_characters.php 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How the query excludes associated records
 - How filters and counts stay aligned
 - What the selected visibility rule is
 - How the server enforces that rule

Response

The query uses NOT EXISTS to exclude any character that already has an active UserCharacters relationship. The same filters are applied to both the list query and the matching-count query, so the results and counts stay consistent. The selected visibility rule is Admin-only access. The server enforces this with `require_role("Admin")` before showing the page.

Supports **bold**, *italic*, [links](url), `code`, etc.

359 chars

Section #5: (2 pts.) Association Assignment

100%

- Goal: show that an authorized role can search for users and project records before applying relationships.
- Expected behavior:
 - User search uses a partial username.
 - Project-record search uses at least one user-friendly value:
 - Identifier
 - Name
 - Title
 - An equivalent project-specific value
 - User search returns no more than 25 matches.
 - Project-record search returns no more than 25 matches.

- An equivalent project-specific value
- User search returns no more than 25 matches.
- Project-record search returns no more than 25 matches.
- The two result sets appear in clear groups or columns.
- Every user result has a checkbox.
- Every project-record result has a checkbox.
- Applying the form creates missing relationships for selected pairs.
- Applying the form removes existing relationships for selected pairs.
- A server-side role check protects the page.
- A server-side role check protects the state-changing request.

🏠 Task #1 (1 pt.) – Assignment Search And Selection Evidence

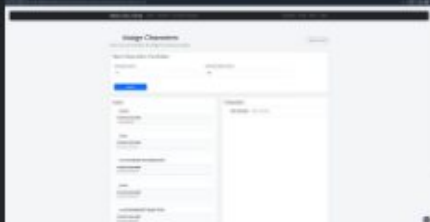
100%

🔗 Combo #1

🖼️ Part 1: Images 100%

📌 Details

- Show one deployed **qa** page with these items visible:
 - User result group
 - Project-record result group
 - Identifying values
 - User checkboxes
 - Project-record checkboxes
- Show both searches returning records that match the submitted values:
 - Partial-username search
 - Project-record search
- Show focused code containing:
 - User search
 - Project-record search
 - User result limit of 25
 - Project-record result limit of 25



Assignment search showing matching users and characters with identifying details and selection checkboxes.



Assignment searches use partial matching and limit both user and character results to 25.

🔗 Part 2: URLs 100%

📌 Details

- Paste the direct GitHub file link for the assignment page.
- Paste the anticipated production URL for the assignment page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/admin/assign_characters.php 🍏

URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/assign_characters.php 🍏

🔗 Part 3: Response 100%

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How the user search builds its result group
 - How the project-record search builds its result group
 - How checkbox names identify selected records
 - How checkbox values identify selected records

Response

The user search uses a partial username match and builds a user result group with identifying information and checkboxes. The character search also uses partial matching and builds a separate character result group. User checkbox names identify the selected user IDs, while character checkbox names identify the selected character IDs. The checkbox values contain the specific record IDs used when applying the selected associations.

Supports **bold**, *italic*, [links](url), ``code``, etc.

433 chars

Task #2 (1 pt.) – Assignment Toggle And Authorization Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show selected user and project-record checkboxes before submission.
- Show the relationship state after one missing pair is created and one existing pair is removed.
- Show one rejected unauthorized assignment request.
- Show focused code containing:
 - Selected-pair processing
 - Missing-pair creation
 - Existing-pair removal
 - Page role check
 - Handler role check



Selected user and character pairs before toggling one existing and one missing relationship.



Toggle created the missing mrc82 relationship and removed the existing michos relationship.



Non-Admin user is denied access to the assignment page on QA.



Assignment page requires the Admin role before users can access or submit changes.

Non-Admin user is denied access to the assignment page on QA.

Assignment page requires the Admin role before users can access or submit changes.



Assignment handler validates the selected user and character IDs before processing pairs.



Each selected user is paired with each selected character and processed by the assignment handler.



Assignment query creates missing relationships and toggles existing relationships active or inactive.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the state-changing assignment handler.
- Paste the anticipated production URL for the assignment page where the action begins.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/admin/assign_characters.php



URL #2

https://mrc82-it202-450-m2026-prod.onrender.com/project/admin/assign_characters.php



Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - How selections become pairs
 - How missing pairs are created
 - How existing pairs are removed
 - How the server limits the workflow to the role selected by the project

Response

The selected user IDs and character IDs are combined so each selected user is paired with each selected character. If a pair does not already exist, the handler creates a new UserCharacters relationship with `is_active` set to 1. If the pair already exists, the handler toggles the existing relationship so it becomes inactive instead of creating a duplicate. The workflow is limited to Admin users because the server requires the Admin role before the assignment page and state-changing request can run.

Section #6: (1 pt.) Public User Profile

100%

- Goal: show safe public user information without exposing private account or authorization data.
- Expected behavior:
 - Association pages display the username.
 - Username links identify the profile by user ID.
 - The profile validates the requested user ID.
 - The profile shows the username.
 - The profile may also show associated-record summaries or other intentionally public project information.
 - Email is not exposed.
 - Roles are not exposed.
 - Internal session details are not exposed.
 - Other private account values are not exposed.

Task #1 (1 pt.) – Public User Profile Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show a deployed qa public profile reached through a username link with these items visible:
 - Username
 - Any additional public project information when used by the project
- Show focused code containing:
 - User-ID validation
 - A query limited to public values
 - Escaped output
- Show that these private values are absent from both the page and query:
 - Email
 - Roles
 - Session data
 - Other private account values



Public profile showing username and saved-character information without private account data.



Public profile validates the requested user ID before loading profile data.



Public profile query loads only public user values and saved-character count without private account data.



Public profile escapes the username before displaying it on the page.

Part 2: URLs 100%

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for the public-profile page.
- Paste the anticipated production URL for one valid public-profile example.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone3/public_html/project/profile.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/profile.php?id=4> 🍏

Part 3: Response 100%

Details

- In 3–5 sentences, explain:
 - Which values are public
 - Which values are excluded
 - How the page safely identifies the requested user

Response

The public profile shows the username, account creation date, saved-character count, and public saved-character information. Private values such as email, password data, roles, and session details are excluded. The requested user is identified by a validated numeric user ID from the URL. The page then loads only the allowed public fields and escapes displayed output before showing it.

Supports **bold**, *italic*, [links](url), `code`, etc.

387 chars

Section #7: (1 pt.) Misc

100%

Task #1 (0.33 pts.) – GitHub Details

100%

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the Milestone 3 pull request.
- The screenshot should show a commit focused on the relationship table.
- The screenshot should show a commit focused on the logged-in user view.
- The screenshot should show a commit focused on the cross-user or unassociated views.
- The screenshot should show a commit focused on the assignment workflow.
- The screenshot should show a commit focused on the public profile.
- The screenshot should show a commit focused on verification or final fixes.



Milestone 3 commits covering relationships, user views, assignment, profiles, and final fixes.

Part 2: URLs 100%

Details

- Paste the pull request URL.
 - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/25> 🍏

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The duration is not tied to the grade.
- Visual graphs are not required.



WakaTime project dashboard showing the repository name and overall coding time.



WakaTime file-level activity showing coding time across project files.

Task #3 (0.33 pts.) – Reflection

100%

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned how to create and manage user-to-character relationships in a database. I also learned how to use filtering, sorting, pagination, and ownership checks.

I learned how to create and manage user-to-character relationships in a database. I also learned how to use filtering, sorting, pagination, and ownership checks.

Supports **bold**, *italic*, [links](url), `code`, etc.

161 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

I would say The easiest part was displaying the saved characters for the logged-in users.

Supports **bold**, *italic*, [links](url), `code`, etc.

89 chars

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The hardest part was updating many of the files from the previous milestones to work with the new features. Like a lot of updates..

Supports **bold**, *italic*, [links](url), `code`, etc.

131 chars

End of Assignment - submission has been recorded.